

Anatomy of an Automated Mail-Server Pipeline

How four small scripts work together to register a domain, spin up a cloud server, wire up DNS, and deploy a fully self-hosted email stack — explained step by step, with diagrams.

`1_provision_vps.py``2_register_domain.py``3_deploy_mailserver.sh``run_all.py`

What it is: a 4-file automation that stands up your own mail server on a DigitalOcean droplet, on a domain bought through Porkbun, running the open-source `docker-mailserver`.

Generated: 29 June 2026 · A plain-English reference to the code you uploaded.

What this program actually does

Running your own email server normally means a dozen fiddly, error-prone manual chores: buying a domain, renting a server, editing DNS, getting a TLS certificate, installing mail software, and proving to the world that you're allowed to send mail. **This pipeline automates that whole sequence.** You end up with a working inbox at `you@yourdomain.xyz`, hosted entirely on hardware you control.

The work is split across **four files**. Three of them each own one stage of the job and can be run on their own; the fourth, `run_all.py`, is an orchestrator that calls the other three in the right order and waits for the slow parts (DNS propagation, the server booting) in between.



1_provision_vps.py

~70 lines · Python

Rents the server. Calls the DigitalOcean API to create a small Ubuntu virtual machine ("droplet"), waits for it to boot, and hands back its public IP address.



2_register_domain.py

~300 lines · Python

Buys the name + wires DNS. Finds and registers a cheap domain on Porkbun (with budget guards and a dry-run), then creates the DNS records that point mail at the server.



3_deploy_mailserver.sh

~95 lines · Bash

Installs the mail stack. Runs *on* the droplet: installs Docker, fetches a Let's Encrypt certificate, and launches `docker-mailserver`.



run_all.py

~150 lines · Python

The conductor. Imports the other scripts as modules and runs the whole thing end-to-end with one command, handling timing and SSH for you.

🛡️ A note on email authentication

A big part of the job is convincing other mail servers that your messages are legitimate and not forged. That's the purpose of the **SPF, DKIM and DMARC** records the pipeline sets up — the same anti-spoofing standards every real mail provider uses. They're explained in their own section later.

How the pieces fit together

Two cloud services do the heavy lifting. Your scripts are just the glue that drives their APIs and ties the result together.

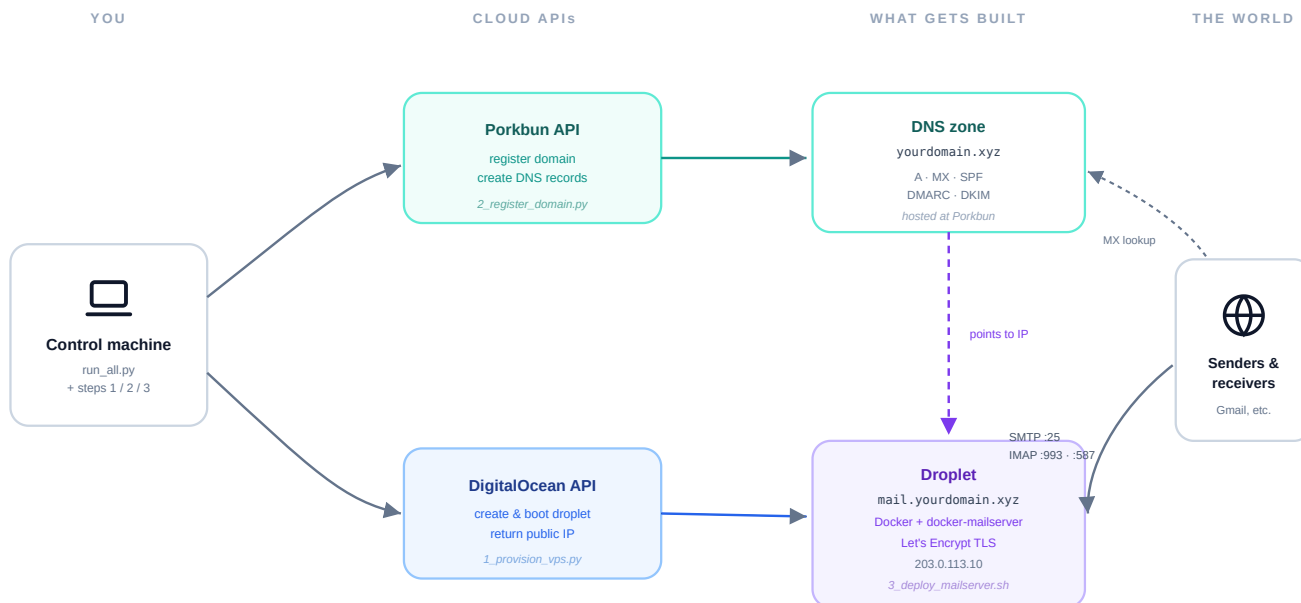


Figure 1 — Your machine drives two cloud APIs; the result is a server plus a DNS zone that points the outside world at it.

The cast of characters

- **DigitalOcean** — the cloud host. Provides the "droplet" (a virtual Linux server) your mail runs on.
- **Porkbun** — the domain registrar. Sells the domain name and hosts its DNS zone.
- **Let's Encrypt** — a free certificate authority. Issues the TLS certificate that encrypts mail connections.
- **Docker** — container runtime. Keeps the mail software neatly packaged on the server.
- **docker-mailserver** — the actual mail stack (Postfix, Dovecot, SpamAssassin...) bundled as one container.

What you need before running

- ✓ A DigitalOcean account + API token (dop_v1_...)
- ✓ A Porkbun account with credit + API key/secret (pk1_ / sk1_)
- ✓ An SSH key uploaded to DigitalOcean (so the orchestrator can log in)
- ✓ Patience for DNS to propagate — and a request to your host to open port 25

VPS ~ \$6 / month

domain < \$5 / year

Two ways to run it

Manual · one stage at a time

Run each script yourself, copying values between them by hand. The numbering hints at the natural reading order:

- 1 provision the VPS → note the IP
- 2 register the domain & set DNS (you feed it the IP)
- 3 SSH in and run the deploy script

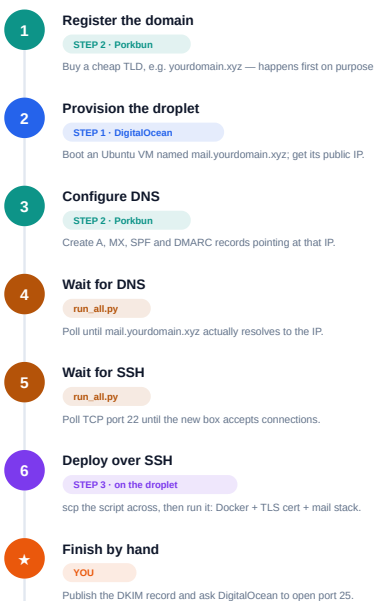
Good for understanding each piece, or recovering when one step needs a retry.

Orchestrated · one command

`run_all.py` does everything, but deliberately **re-orders** the steps — the domain is registered *before* the server exists:

```
$ export
DIGITALOCEAN_API_TOKEN=dop_v1...
$ export DO_SSH_KEY_IDS=12345678
$ python run_all.py
```

Why the reorder? So the server can be *named* after the domain and DNS can be live before the certificate is requested. More on this shortly.



The orchestrated sequence, narrated

Reading the timeline on the left from top to bottom:

- **1 — Domain first.** `step2.select_and_register()` buys the name. It must come first so the next step can name the server after it.
- **2 — Server.** `step1.provision_vps(name="mail.<domain>")` boots the droplet and returns its IP.
- **3 — DNS.** `step2.configure_dns(domain, ip)` points the records at the fresh IP.
- **4 — Wait for DNS.** The script polls until the name resolves (up to 15 min), because the certificate step will fail otherwise.
- **5 — Wait for SSH.** It checks port 22 until the box is reachable (up to 3 min).
- **6 — Deploy.** It copies `3_deploy_mailserver.sh` over and runs it remotely.
- **★ — Manual finish.** Two things can't be automated: publishing DKIM and opening outbound port 25.

Figure 2 — The colour of each node shows which underlying script does the work; note how Step 2 runs twice, on either side of Step 1.

Renting a server in code

This is the simplest script. Its single job is to ask DigitalOcean for a virtual machine and not return until that machine is alive and has an IP address. Everything happens through one function, `provision_vps()`.

What it builds (the defaults)

SETTING	VALUE
size	s-1vcpu-1gb — the ~\$6/mo tier
image	ubuntu-22-04-x64
region	nyc3 (New York)
ssh_keys	optional — attached so login needs no password

The shape of the work

- **Create.** POST the droplet spec; a 202 Accepted means "queued, building".
- **Poll.** Every 10 seconds, check the droplet's status — up to a 5-minute ceiling.
- **Extract.** Once status flips to active, pull the public IPv4 address out of the response.
- **Return.** Hand that IP back to the caller (or print it, if run standalone).

The essence, in code

```
# build the request, optionally attaching SSH keys
payload = {"name": name, "size": "s-1vcpu-1gb",
           "image": "ubuntu-22-04-x64", "region": "nyc3"}
if ssh_key_ids:
    payload["ssh_keys"] = ssh_key_ids

resp = requests.post(BASE_URL, json=payload, headers=_headers())
# 202 = accepted. Now poll /droplets/{id} until it reports "active",
# then read the public IP out of droplet["networks"]["v4"].
```

🔑 Why SSH keys matter here

If you attach an SSH key, the orchestrator can later log in unattended. If you *don't*, DigitalOcean emails you a root password instead — fine for manual use, but it breaks the one-command flow, since `run_all.py` needs key-based login to deploy Step 3.

Buying the name — carefully

This is the largest and most defensive script, because it spends real money. Before it ever charges your Porkbun credit, a request passes through a chain of checks — a **safety funnel** — and you have to type yes at the end.

The safety funnel

```
validate_credentials()      # 1. POST /ping - are the API keys valid?
domain, price = choose_domain() # 2. checkDomain - available? not premium? ≤ $5?
dry_run_register(domain, price) # 3. create(dryRun) - "wouldSucceed"? funds OK?
# 4. you type 'yes' to confirm the charge -----
register(domain, cents)      # 5. create(real) + a unique Idempotency-Key
verify(domain)               # 6. confirm it now lives in your account
```

Finding a domain

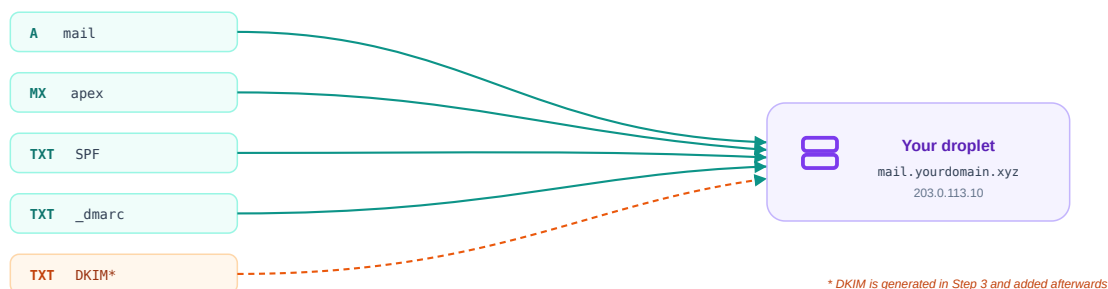
You can name one outright (DESIRED_DOMAIN), or let it suggest: it walks a list of cheap TLDs (.xyz .click .online...), checks each for availability and price, and offers the ones under budget. Because Porkbun rate-limits these checks, it deliberately pauses ~10 s between names.

The dry run is the key guard

Porkbun supports a dryRun registration that returns wouldSucceed, your balance, and whether funds and any monthly spend cap are OK — **without charging**. The script refuses to proceed unless the dry run says it would work.

Then it wires up DNS

With the IP in hand, `configure_dns()` creates four records (a fifth, DKIM, comes later). Together they tell the internet *where* your mail lives and *who* is allowed to send it.



TYPE	HOST	VALUE (EXAMPLE)	WHAT IT'S FOR
A	mail	203.0.113.10	Maps <code>mail.yourdomain.xyz</code> to the server's IP.
MX	@ (apex)	<code>mail.yourdomain.xyz</code> (prio 10)	Tells other servers "deliver mail for this domain here".
TXT	@ (SPF)	<code>v=spf1 mx ~all</code>	Declares which servers may send mail as you.
TXT	_dmarc	<code>v=DMARC1; p=quarantine; rua=mailto:postmaster@...</code>	Sets the policy for failures and where to send reports.
TXT	mail._domainkey	(added in Step 3)	DKIM — the public half of the signing key.

Figure 3 — Four records are created here; the DKIM record can only be added once Step 3 generates the key.

SPF, DKIM & DMARC, briefly

These three DNS records are what stop your mail from landing in spam. Each answers a different question a receiving server asks about an incoming message.

SPF — "is this server allowed to send for the domain?"

A list, published in DNS, of who may send mail using your domain name. The value `v=spf1 mx ~all` means: *the hosts in my MX record are allowed; treat anything else as suspect*. It stops strangers from sending mail that claims to be from your domain.

DKIM — "was this message tampered with in transit?"

Your server signs every outgoing message with a private key; the matching public key sits in the `mail._domainkey` DNS record. Receivers verify the signature. This is why DKIM is added **last** — the key only exists after the mail server is installed and generates it in Step 3.

DMARC — "what should I do if the checks fail, and who do I tell?"

The policy that ties SPF and DKIM together. Here it's set to `p=quarantine` — mail that fails the checks should be treated as spam — and it asks receivers to mail aggregate reports to `postmaster@` your domain so you can see how your mail is being judged.

Putting it together

A receiving server asks all three: *Is the sending server authorised (SPF)? Is the signature valid (DKIM)? And given the answers, what's the owner's wishes (DMARC)?* Setting all three correctly is the difference between a mail server that's trusted and one that's silently dropped — which is exactly why the pipeline bothers to configure them.

Standing up the mail stack

Unlike the others, this is a **Bash script that runs on the droplet itself**, as root. It takes the domain (and optionally a first mailbox's address and password) and turns a blank Ubuntu box into a working mail server. It runs top to bottom:

- **Install Docker.** Adds Docker's apt repository and installs the engine + compose plugin.
- **Get a TLS certificate.** Runs `certbot --standalone` for `mail.<domain>`. This needs port 80 free and DNS already resolving — hence all the waiting earlier.
- **Write `docker-compose.yml`.** Declares the `docker-mailserver` container, its ports, storage volumes, and feature switches.
- **Launch.** `docker compose up -d` starts it in the background.
- **Create a mailbox & DKIM key.** Adds your first user (if given) and generates the DKIM signing key with the bundled `setup.sh`.

Ports the container opens

25	SMTP — server-to-server mail
587	Submission — you sending mail
143	IMAP — reading mail
993	IMAP over TLS — reading, encrypted

Feature switches

SpamAssassin on

Fail2ban on

ClamAV off

TLS = Let's Encrypt

The container definition (abridged)

```
services:
  mailserver:
    image: ghcr.io/docker-mailserver/docker-mailserver:latest
    hostname: mail.<domain>
    ports: [ "25:25", "143:143", "587:587", "993:993" ]
    environment:
      - ENABLE_SPAMASSASSIN=1
      - ENABLE_CLAMAV=0
      - ENABLE_FAIL2BAN=1
      - SSL_TYPE=letsencrypt # reuses the certbot cert mounted read-only
```

🕒 The certificate is the fragile step

Let's Encrypt proves you control the name by connecting back to it over HTTP. If DNS hasn't propagated yet, or port 80 is blocked, `certbot` fails — so the script warns you and says it can simply be re-run once `mail.<domain>` resolves. This is the whole reason the orchestrator waits for DNS before reaching this step.

What happens when mail arrives

Once everything is wired up, here's the journey of a single incoming message — and how each record and port from the earlier steps comes into play.

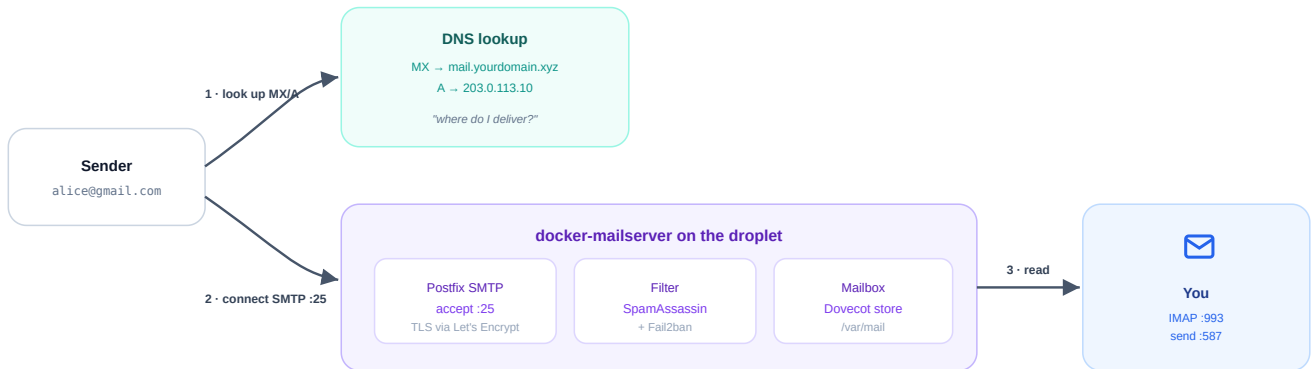


Figure 4 — A sender finds you via DNS, delivers over SMTP, the stack filters and stores the message, and you read it over IMAP.

The conductor: run_all.py

The orchestrator is mostly **glue and patience**. It loads steps 1 and 2 as Python modules (so it can call their functions directly), then drives the sequence and handles the slow, real-world waits:

- `wait_for_dns()` — re-checks every 30 s, up to 15 minutes, until the name resolves to the new IP.
- `wait_for_ssh()` — polls port 22, up to 3 minutes, until the box accepts logins.
- `deploy_remote()` — `scps` the deploy script across, then runs it over `ssh`.

If the remote deploy fails, it prints the exact two commands to finish by hand, so nothing is lost.

Why register the domain *first* ?

This is the cleverest design choice. By buying the domain before creating the server, the orchestrator can:

- **Name the server** `mail.<domain>`, so DigitalOcean sets a matching reverse-DNS (PTR) record — which other mail servers check for trust.
- **Set DNS early**, giving it the maximum time to propagate before the certificate step needs it to be live.

What you still have to do by hand

Two things genuinely can't be automated — the scripts are honest about this and print reminders at the end.

1 · Publish the DKIM record

Step 3 prints the generated key. You copy it into a new TXT record at `mail._domainkey.<domain>` in Porkbun. Until you do, outgoing mail isn't signed.

2 · Open outbound port 25

Most clouds — DigitalOcean included — block outbound port 25 by default to fight spam. You must open a support ticket to enable sending. *Receiving* mail works without it.

The guardrails built into the code

Because this pipeline spends money and touches live infrastructure, the scripts are written defensively. Worth knowing they're there:

- ✓ **Budget cap.** Domains over `MAX_PRICE_USD` (\$5) are skipped or flagged.
- ✓ **Dry run before spending.** The registration is simulated first; it won't buy unless the simulation says it would succeed.
- ✓ **Explicit confirmation.** You must type yes before any charge.
- ✓ **Idempotency key.** A unique key on the purchase request prevents accidental double-registration on a retry.
- ✓ **Timeouts everywhere.** Every "wait" loop — provisioning, DNS, SSH — gives up gracefully rather than hanging forever.
- ✓ **Graceful fallbacks.** A failed remote deploy prints the manual commands to finish the job.

Things that commonly trip it up

- **Port 25 still blocked** — you can receive but not send until the host opens it.
- **DNS not propagated** — the certbot step fails; just re-run once the name resolves.
- **Premium TLD** — some names can't be registered via API; buy them on the website instead.
- **Missing API permissions** — DNS edits need "API Access" enabled for the domain in Porkbun.

In one sentence: register a cheap domain, rent a small server, point DNS at it, wait for the internet to catch up, then install a containerised mail stack on it — with money-spending steps guarded by dry-runs and confirmations, and two final touches left for you.